ZX SPECTRUM MANIC MINER

Guida Tecnica Completa, Architettura del Progetto e Analisi dei Sorgenti Z80

Studio approfondito del motore di gioco, fisica, collision detection pixel-perfect, IA guardiani e rendering VRAM

Documento di Riferimento per Ingegneri Software, Retrogamer e Sviluppatori Assembly

Codice Sorgente Target: Sinclair ZX Spectrum 48K (Z80A @ 3.50 MHz) - Compilatore: sjasmplus

SPECIFICHE TECNICHE DELL'ARCHITETTURA HARDWARE E SOFTWARE

Parametro	Valore / Indirizzo	Descrizione Architetturale
Target Hardware	ZX Spectrum 48K	CPU Zilog Z80A @ 3.50 MHz, ULA custom, 48KB RAM, 16KB ROM
Entry Point	0x5DAD (org 23981)	Indirizzo di avvio del programma, Stack pointer inizializzato a $FDF0
VRAM Pixel Buffer	$4000..$57FF (6144 B)	Matrice 256x192 pixel a 3 terzi con scanline interlacciate
VRAM Attribute Buffer	$5800..$5AFF (768 B)	Matrice 32x24 celle colore (FLASH, BRIGHT, PAPER, INK)
Interrupt Mode	IM 2 (Vettore $FE00)	Interrupt a 50Hz sincronizzato con il vertical blank della ULA
Dimensione Caverne	1024 Byte (1 KB)	20 caverne da 1024 byte ciascuna (Layout, Tile, Nemici, Oggetti, Portale)
Formato Punteggio	BCD 3 Byte (6 Cifre)	Aritmetica decimale codificata in binario tramite istruzione DAA

PREMESSA: SCOPO E STRUTTURA DEL DOCUMENTO

Il presente documento costituisce la guida tecnica integrale e autosufficiente del progetto Manic Miner per Sinclair ZX Spectrum 48K. Il manuale è stato redatto analizzando riga per riga l'intera base di codice Assembly Z80 del repository, con l'obiettivo di fornire una documentazione esaustiva che consenta a qualsiasi programmatore di comprendere a fondo il funzionamento, l'architettura logica, la gestione della memoria, la fisica del movimento, il motore di rendering e le strategie di ottimizzazione implementate nel gioco.

Tutti i nomi di file, funzioni, etichette (labels), variabili e costanti corrispondono esattamente a quanto definito nei sorgenti del progetto. I concetti teorici sono corredati da schemi a blocchi, tabelle riassuntive e porzioni di codice commentate che illustrano le scelte implementative e i workaround tecnici resi necessari dai vincoli dell'hardware target.

INDICE STRUTTURA DELLA GUIDA TECNICA

PREMESSA SCOPO E STRUTTURA DEL DOCUMENTO 2
INDICE STRUTTURA DELLA GUIDA TECNICA 3
1 PANORAMICA DEL GIOCO E MECCANICHE FONDAMENTALI 5
1.1 Obiettivo del Gioco e Condizioni di Vittoria 5
1.2 Meccaniche Principali e Dinamiche di Gioco 5
1.3 Struttura Generale di una Partita 5
1.4 Gestione dei Livelli e Flusso delle Caverne 5
1.5 Regole di Gioco, Vite, Punteggio e Game Over 6
2 ARCHITETTURA DEL PROGETTO E ORGANIZZAZIONE DEL CODICE 7
2.1 Struttura delle Cartelle e Mappa dei File 7
2.2 Funzione e Responsabilita dei Principali Moduli 7
2.3 Grafo delle Dipendenze e Flusso dei Dati 7
2.4 Assemblaggio, Mappa della Memoria RAM e Target Hardware 8
3 GESTIONE DEL GIOCATORE (MINER WILLY) 9
3.1 Strutture Dati e Variabili di Stato del Giocatore 9
3.2 Sistema di Movimento Orizzontale a Terra e in Volo 9
3.3 Modello Fisico del Salto Parabolico e Caduta 9
3.4 Limiti di Schermo, Clamp e Coordinate Pixel 9
3.5 Macchina a Stati del Giocatore e Condizioni di Morte/Respawn 10
4 MOTORE FISICO E RILEVAMENTO DELLE COLLISIONI 11
4.1 Decompressione del Layout e Mappatura delle Celle (unpacked_layout) 11
4.2 Rilevamento Collisioni con Pavimenti e Piattaforme (Phys_CheckSolid) 11
4.3 Collisioni con Muri e Soffitti (CheckWillyWallCollision, CheckWallCell) 11
4.4 Meccanica Dinamica dei Pavimenti Sgretolabili (Phys_CheckCrumblingTrigger) 11
4.5 Modello Matematico delle Rampe Inclinate (GetRampY) 11
4.6 Nastri Trasportatori e Logica di Cattura (CheckWillyOnConveyor) 12
4.7 Rilevamento Ostacoli Letali (Phys_CheckNastyCollision) 12
5 SISTEMA DEI NEMICI E INTELLIGENZA ARTIFICIALE 13
5.1 Tabella dei Nemici Attivi (ActiveEnemies) e Struttura dello Slot 13
5.2 Guardiani Orizzontali: Ciclo Animazione a 8 Frame e Inversione 13
5.3 Guardiani Verticali: Pattugliamento Lineare e Velocita Frazionaria 13
5.4 Comportamento Speciale del Boss Eugene (Caverna 04) 13
5.5 Meccaniche del Boss Kong Beast e Interruttori (Caverne 07 e 11) 13
5.6 Collision Detection Pixel-Perfect Bit-a-Bit (CheckCollision) 14
6 OGGETTI, PUNTEGGIO E MECCANICHE DI GAMEPLAY 15
6.1 Struttura e Caricamento degli Oggetti della Caverna 15
6.2 Animazione Ciclica del Colore degli Item (Items_Animate) 15
6.3 Raccolta Oggetti e Sistema di Scoring BCD a 6 Cifre 15
6.4 Gestione dell'High Score Persistente 15
6.5 Portale di Fine Livello e Svuotamento Riserva d'Aria (Air_DepleteCountdown) 15
6.6 Sequenza di Game Over (Piede) e Schermata di Vittoria (Maria) 15
7 STRUTTURA DATI DELLE CAVERNE (1024 BYTE STANDARD) 16
7.1 Mappa Completa dei 1024 Byte della Caverna 16
7.2 Caricamento, Inizializzazione e Transizione tra Stanze 16
7.3 Analisi Dettagliata delle 20 Caverne del Gioco 16
8 GRAFICA, VRAM E RENDERING FLICKER-FREE 18
8.1 Organizzazione e Geometria della VRAM dello ZX Spectrum ($4000..$5AFF) 18
8.2 Calcolo Matematico degli Indirizzi Video e Scanline (Gfx_GetScreenAddr) 18
8.3 Rendering degli Sprite e Ricostruzione dello Sfondo (Gfx_GetBackgroundByte) 18
8.4 Pipeline di Rendering a Due Passate e Risoluzione Sovrapposizioni 18
8.5 HUD, Barra dell'Aria Dinamica e Animazione delle Vite 19
9 GAME LOOP, SINCRONIZZAZIONE E GESTIONE INPUT 20
9.1 Ciclo Principale di Gioco (Main_loop) e Sequenza Rigorosa delle Fasi 20
9.2 Sincronizzazione Hardware Vertical Blank via Interrupt IM2 a 50Hz 20
9.3 Scansione della Tastiera, Matrice delle Porte I/O Spectrum 20
9.4 Coyote Time, Jump Buffer e Gestione Debounce per Controlli Fluidi 20
10 OTTIMIZZAZIONI Z80 E TECNICHE A BASSO LIVELLO 22
10.1 Tecniche di Risparmio di Memoria e Compattazione Dati 22
10.2 Ottimizzazioni di Velocita nei Calcoli Critici a 16-Bit 22
10.3 Gestione Atomica della VRAM per Eliminazione Sfarfallio 22

10.4 Aritmetica BCD Hardware e Lookup Table 22
11 PORZIONI DI CODICE SIGNIFICATIVE COMMENTATE 23
11.1 Il Game Loop Principale (manicminer.asm: Main_loop) 23
11.2 Movimento del Giocatore e Cattura del Conveyor (player.asm: MovePlayer) 23
11.3 Collision Detection Pixel-Perfect Bit-a-Bit (enemy.asm: CheckCollision) 24
11.4 Aritmetica BCD per lo Scoring (items.asm: Score_AddItemPoints) 24
12 DIAGRAMMI DI FLUSSO E SCHEMI ARCHITETTURALI 26
12.1 Flusso del Game Loop Principale 26
12.2 Macchina a Stati di Miner Willy 26
12.3 Pipeline di Collision Detection Pixel-Perfect 27
13 ANALISI CRITICA: WORKAROUND, LIMITAZIONI E PUNTI DELICATI 28
13.1 Gestione dell'Attribute Clash dello ZX Spectrum 28
13.2 Risoluzione del Glitch di Aggancio Rampe dal Basso (ramp_entry_guard) 28
13.3 Preservazione dei Registri in Funzioni di Calcolo Fisico 28
13.4 Gestione dello Stack Pointer (ld sp, $fdf0) per Evitare Stack Drift 28
13.5 Limiti di Scalabilita del Modello a 1024 Byte 28

1 PANORAMICA DEL GIOCO E MECCANICHE FONDAMENTALI

1.1 Obiettivo del Gioco e Condizioni di Vittoria

Manic Miner e' uno dei capostipiti del genere platform a schermata fissa (single-screen platformer), sviluppato originariamente da Matthew Smith nel 1983 per l'home computer Sinclair ZX Spectrum 48K. L'obiettivo primario del giocatore e' guidare il protagonista, Miner Willy (il minatore Willy), attraverso una sequenza di 20 caverne sotterranee colme di pericoli, trappole letali e creature ostili.

In ciascuna caverna, il giocatore deve tassativamente raccogliere tutti gli oggetti scintillanti (chiavi/tesori) sparsi per la schermata. La raccolta dell'ultimo oggetto presente nella stanza scatena l'attivazione del Portale d'Uscita (Portal), che inizia a lampeggiare con un ciclo cromatico arcobaleno. Il completamento del livello avviene nel momento esatto in cui la bounding box del giocatore entra in collisione con il portale attivo:

> **Condizione di Avanzamento Livello:** Raccolta di tutti gli oggetti (num_room_items == 0) e sovrapposizione fisica di Willy con il portale (CheckPortalTransition attiva). Al passaggio, l'aria residua viene convertita in punti bonus.

> **Condizione di Vittoria Globale:** Superamento della ventesima ed ultima caverna (Stanza 19: The Final Barrier). Al raggiungimento del portale finale, si attiva la sequenza cinematica di vittoria (TriggerVictory) in cui Willy raggiunge la moglie Maria (la 'VecchiaSprite') nella Master Bedroom.

1.2 Meccaniche Principali e Dinamiche di Gioco

Il gameplay di Manic Miner si distingue per una fisica deterministica, inflessibile e priva di qualsiasi inerzia residua, richiedendo una precisione millimetrica nel tempismo e nella posizione di salto.

> **Movimento Orizzontale a Terra:** Willy si muove a terra a una velocita fissa di 2 pixel per frame (WILLY_WALK_SPEED = 2). La pressione dei tasti direzionali sposta immediatamente il player senza fasi di accelerazione o decelerazione.

> **Salto Parabolico a Traiettoria Rigida:** A differenza dei moderni platform con salto variabile, in Manic Miner il salto ha un arco e una traiettoria immutabili e non correggibili a mezz'aria. Una volta iniziato un salto direzionale, Willy continuera inesorabilmente a volare nella direzione iniziale (jump_dir) per l'intera durata del salto.

> **Nastri Trasportatori (Conveyor):** Piattaforme speciali animate che trascinano forzatamente Willy a sinistra o a destra. Se Willy cerca di opporsi camminando contro il nastro, puo avanzare temporaneamente, ma non appena il tasto direzionale viene rilasciato, Willy viene 'catturato' (conveyor_caught = 1) e trascinato ineluttabilmente verso il bordo del nastro.

> **Pavimenti Sgretolabili (Crumbling Floor):** Piattaforme di roccia friabile a 8 stadi di dissoluzione. Ogni frame in cui Willy calpesta un blocco sgretolabile, la texture del blocco scende verso il basso di 1 scanline. Al raggiungimento dell'ottavo stadio (8 pixel di dissoluzione), il blocco collassa completamente trasformandosi in spazio vuoto (Background) e facendo precipitare Willy.

> **Rampe Inclinate a Pendenza Analitica:** Piani inclinati a 45 gradi sui quali Willy sale e scende con continuita pixel-by-pixel, agganciando la quota calcolata analiticamente dalla formula di pendenza.

> **Celle Letali Fisse (Nasty Tiles):** Ostacoli ambientali fissi come punte rocciose, stalattiti, pozze di acido, fiamme o ragni statici. Il minimo contatto tra il bounding box di Willy e una cella Nasty provoca la morte immediata del protagonista.

> **Guardiani Mobili (Nemici):** Creature che pattugliano la caverna secondo percorsi ciclici prefissati orizzontali o verticali, con animazioni sincronizzate e rilevamento delle collisioni pixel-perfect (bit a bit).

1.3 Struttura Generale di una Partita

L'architettura del flusso di gioco segue un automa a stati finiti rigorosamente gerarchico:

Stato di Gioco	Routine Principale	Eventi di Transizione e Condizioni
Title Screen	ShowTitleScreen	Mostra logo e grafica del titolo con animazione colore ciclica. Attende la pressione del tasto ENTER per azzerare lo stato e avviare la Caverna 00.
In-Game Loop	Main_loop	Esegue a 25 FPS il ciclo sincrono: rendering nemici, disegno Willy, aggiornamento aria, audio, lettura input, fisica, collisioni e raccolta item.
Morte / Respawn	.willy_dies	Decrementa willy_lives. Se vite > 0, esegue una pausa a schermo nero di 0.5s, ripristina gli item della stanza e ricarica Willy allo spawn point.
Game Over	TriggerGameOver	Scatenato all'esaurimento delle vite (willy_lives == 0). Aggiorna l'High Score, esegue l'animazione del Piede gigante sul piedistallo con audio beeper e torna a Title Screen.
Vittoria Finale	TriggerVictory	Scatenato al completamento della Stanza 19. Disegna Maria che cammina verso Willy, anima 'CONGRATULATIONS!' e chiede 'PLAY AGAIN? Y/N'.

1.4 Gestione dei Livelli e Flusso delle Caverne

Il gioco comprende 20 caverne numerate da 00 a 19, ciascuna definita da un blocco dati monolitico di esattamente 1024 byte (1 KB).
I puntatori a tutte le stanze sono registrati nella tabella RoomsListTable:

> **Sequenza Caverne (00..19):** 00: Central Cavern, 01: The Cold Room, 02: The Menagerie, 03: Abandoned Uranium Workings, 04: Eugene's Lair, 05: Processing Plant, 06: The Vat, 07: Miner Willy meets the Kong Beast, 08: Wacky Amoebatrons, 09: The Endorian Forest, 10: Attack of the Mutant Telephones, 11: Return of the Alien Kong Beast, 12: Ore Refinery, 13: Skylab Landing Bay, 14: The Bank, 15: The Sixteenth Cavern, 16: The Warehouse, 17: Amoebatrons' Revenge, 18: Solar Power Generator, 19: The Final Barrier.

1.5 Regole di Gioco, Vite, Punteggio e Game Over

Le regole di ingaggio e i parametri di sopravvivenza sono rigidamente codificati:

> **Riserva di Vite:** Il giocatore inizia la partita con 7 vite (INITIAL_LIVES = 7 in constants.asm). Il contatore e' memorizzato nella variabile willy_lives.

> **Altezza di Caduta Letale (Fall Death):** Willy puo cadere in sicurezza per un massimo di 40 pixel verticali (MAX_SAFE_FALL_HEIGHT = 40, pari a 5 blocchi da 8 pixel). Se l'atterraggio avviene dopo un dislivello superiore a 40 pixel rispetto alla quota di inizio caduta (jump_start_y), Willy soccombe all'impatto.

> **Riserva d'Aria e Asfissia:** Ogni caverna possiede una riserva d'aria visualizzata graficamente dalla barra AIR a riga 17 (224 pixel). L'aria si consuma al ritmo di 1 pixel ogni N tick (Phys_UpdateAir). Se l'aria si esaurisce (air_pixels == 0), Willy muore all'istante per soffocamento.

> **Sistema di Punteggio BCD:** Il punteggio e' gestito in memoria in formato BCD (Binary Coded Decimal) a 3 byte (6 cifre decimali 000000..999999). Ogni oggetto raccolto assegna 100 punti. Durante la transizione di fine livello, ogni pixel di aria residua viene convertito in 1 punto supplementare.

2 ARCHITETTURA DEL PROGETTO E ORGANIZZAZIONE DEL CODICE

2.1 Struttura delle Cartelle e Mappa dei File

Il progetto e' organizzato secondo una struttura modulare pulita, dove ogni modulo Assembly Z80 racchiude un sottosistema funzionale ben definito:

Percorso File	Modulo / Funzione	Descrizione e Contenuto Tecnico
manicminer.asm	Main & Flow Control	Entry point del programma ($5DAD), loop principale, gestione transizioni stanze, game over, victory screen e salvataggio snapshot .SNA.
constants.asm	Costanti Globali	Indirizzi ROM/RAM Spectrum, costanti colore, flag interrupt, UDG, costanti stanza e ID sprite.
settings.asm	Flag di Compilazione	Interruttori condizionali IF/ENDIF (musica, effetti, velocita game loop, collision death, debug bars).
physics_config.asm	Parametri Fisica	Configurazione salto di default, gravita, coyote time, jump buffer frames, hitbox inset.
interrupts.asm	Gestione Interrupt IM2	Configurazione tabella e vettore IM 2 ($FE00/$FDFD), ISR a 50Hz per incremento counter.
graphics.asm	Motore Grafico & VRAM	Geometria VRAM Spectrum, disegno layout, rendering pixel-perfect sprite 16x16, HUD, air bar, erase e background restore.
player.asm	Logica Giocatore	Movimento orizzontale Willy a terra/aria, clamp limiti, animazione gambe, spawn da stanza e cattura conveyor.
physics.asm	Motore Fisico	Gravita modulare, salto, atterraggio, crumbling floors a 8 stadi, rampe analitiche, collisioni wall e nasty.
enemy.asm	Sistema Nemici & IA	Tabella ActiveEnemies (8 slot x 11B), guardiani orizzontali/verticali, Eugene, Kong Beast, interruttori e collisioni bit-a-bit.
items.asm	Oggetti & Punteggio	Parsing item stanza (OFFSET_ITEMS), animazione colore $8F63, raccolta bounding box, BCD score e high score.
audio.asm	Beeper Sound Engine	Effetti sonori frame-by-frame sul buzzer porta $FE (salto, raccolta item) e sintetizzatore musicale a 1 bit.
resources.asm	Gestore Risorse	Inclusione di tutte le 20 caverne (rooms/*.asm), sprite grafiche dei personaggi e font buffer UDG.
data/datavariable.asm	RAM Globale Dinamica	Allocazione di tutte le variabili di stato, posizioni, buffer di layout (unpacked_layout, items_grid, crumbling_grid).
rooms/*.asm (20 file)	Layout Caverne (1KB)	File sorgenti individuali delle 20 caverne, ciascuno contenente esattamente 1024 byte di dati di livello.

2.2 Funzione e Responsabilita dei Principali Moduli

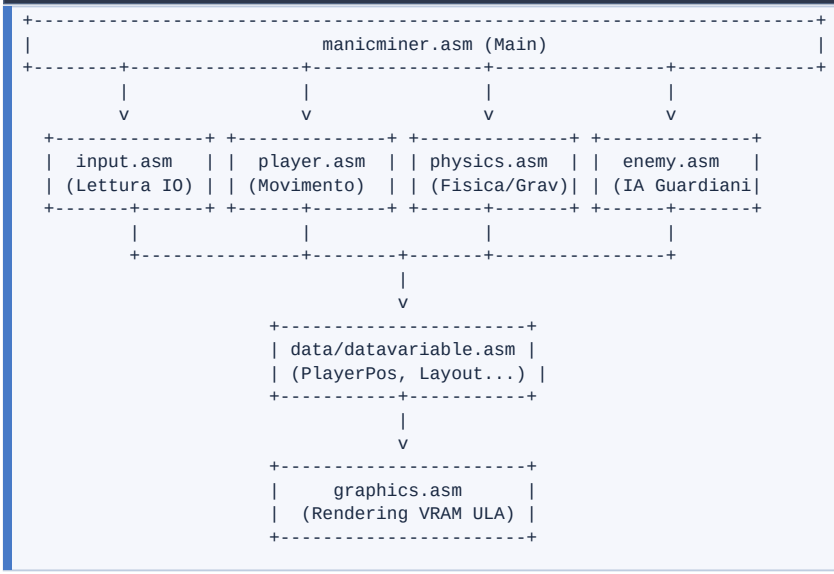
Ciascun file .asm implementa uno strato architetturale rigorosamente isolato:

- **manicminer.asm (Core):** Inizializza l'hardware (imposta stack pointer a $FDF0, apre il canale video 2 della ROM con OPENCHAN, inizializza UDG a $8400, pulisce lo schermo e attiva IM2). Orchestratore del game loop.
- **graphics.asm (Rendering):** Contiene la logica matematica di conversione tra coordinate cartesiane pixel (X, Y) e indirizzi fisici non lineari della VRAM dello ZX Spectrum. Implementa il rendering atomico privo di sfarfallio (Flicker-Free).
- **physics.asm (Physics Engine):** Decomprime il layout in RAM e calcola ad ogni frame la traiettoria di salto, la velocita verticale, l'aggancio millimetrico su pavimenti e rampe e la dissoluzione dei pavimenti sgretolabili.
- **enemy.asm (AI & Guardiani):** Aggiorna ad ogni frame i frame di animazione e le posizioni dei guardiani attivi. Contiene la celebre routine di collisione pixel-perfect bit-a-bit tra lo sprite di Willy e la grafica dei guardiani.

2.3 Grafo delle Dipendenze e Flusso dei Dati

Il flusso di comunicazione tra i moduli si articola su una memoria centrale condivisa (definita in data/datavariable.asm) senza overhead di stack o passaggio di parametri complessi:

DIAGRAMMA DELLE DIPENDENZE E FLUSSO DATI DEL MOTORE



2.4 Assemblaggio, Mappa della Memoria RAM e Target Hardware

Il codice viene compilato tramite l'assemblatore sjasmplus producendo un'immagine di memoria completa da 48KB e uno snapshot avviabile in formato .SNA:

Intervallo Indirizzi	Dimensione	Funzione e Contenuto in Memoria
$0000..$3FFF	16384 B (16KB)	ROM Standard ZX Spectrum (Tabelle matematiche, font ASCII base, routine ROM)
$4000..$57FF	6144 B (6KB)	VRAM Bitmap Pixel ZX Spectrum (256x192 pixel, visualizzata dalla ULA)
$5800..$5AFF	768 B	VRAM Color Attributes (32x24 caratteri a 8x8 con attributi FLASH, BRIGHT, PAPER, INK)
$5B00..$5DAC	685 B	Buffer di sistema ZX Spectrum e variabili BASIC
$5DAD..$E000	~33 KB	Codice Sorgente Eseguibile Manic Miner (Tutti i moduli .asm compilati)
$E000..$F7FF	6144 B	Dati Statici Caverne (20 stanze x 1024 byte ciascuna = 20480 byte complessivi)
$F800..$FDFF	1536 B	Buffer RAM Dinamici (unpacked_layout, items_grid, crumbling_grid, font_buffer)
$FE00..$FEFF	257 B	Tabella di Vettori Interrupt IM 2 (riempita con il byte $FD)
$FDF0	Puntatore	Stack Pointer (SP) hardware inizializzato all'avvio del gioco

3 GESTIONE DEL GIOCATORE (MINER WILLY)

3.1 Strutture Dati e Variabili di Stato del Giocatore

Tutte le proprieta dinamiche di Miner Willy sono allocate in variabili globali a 1 o 2 byte in RAM (data/datavariable.asm):

Variabile	Tipo / Dim.	Valori Ammessi	Funzione Logica nel Motore
PlayerPos	Byte x 2	B=Y (0..127), C=X (5..245)	Coordinate cartesiane attuali in pixel di Willy (Top-Left dello sprite).
PlayerPosOld	Byte x 2	B=OldY, C=OldX	Coordinate del frame precedente, utilizzate per cancellare lo sprite.
player_drawn_pos	Byte x 2	X, Y ($FF se invalido)	Coordinate effettive renderizzate a video nell'ultimo frame.
direction	Byte	1 = Sinistra, 2 = Destra	Orientamento orizzontale dello sprite e della camminata.
body_state	Byte	0, 1, 2	Frame corrente dell'animazione ciclica delle gambe di Willy.
on_ground	Byte	1 = a terra, 0 = in aria	Flag di stato: indica se Willy poggia su solido o e' in volo.
velocity_y	Signed Byte	-12 .. +6	Velocita verticale istantanea (positiva verso l'alto, negativa in caduta).
jump_dir	Byte	0=Vert, 1=SX, 2=DX	Direzione di volo bloccata all'inizio del salto (inalterabile in aria).
jump_start_y	Byte	0..127	Quota Y di partenza del salto/caduta per calcolare il dislivello di morte.
jump_frame	Byte	0..255	Contatore incrementale dei frame di volo per gravita frazionaria.
coyote_timer	Byte	0..4	Timer Coyote Time: permette il salto per 4 frame dopo aver lasciato un bordo.
jump_pressed	Byte	0..5	Jump Buffer: conserva la pressione del tasto Z poco prima di toccare terra.
conveyor_caught	Byte	0 o 1	Flag di cattura nastro: impedisce a Willy di opporsi dopo aver rilasciato il tasto.
willy_safety_timer	Byte	0..255 (frame)	Timer di invulnerabilita temporanea post-respawn contro i guardiani.

3.2 Sistema di Movimento Orizzontale a Terra e in Volo

La routine MovePlayer (in player.asm) gestisce lo spostamento orizzontale di Willy analizzando la maschera di input D generata da Input_CheckCtrl (bit 0 = Sinistra, bit 1 = Destra, bit 2 = Salto):

- **Movimento a Terra (on_ground == 1):** Se Willy e' a terra, lo spostamento orizzontale corrisponde a player_speed (2 pixel per frame). L'animazione delle gambe cicla sequenzialmente attraverso body_state (0 -> 1 -> 2 -> 0).
- **Movimento in Aria (on_ground == 0):** Durante il salto o la caduta, la direzione e' rigidamente vincolata a jump_dir. Il codice ignora completamente i tasti premuti in quel frame. La velocita orizzontale in volo alterna 2 e 3 pixel per frame (esaminando bit 0 di jump_frame) garantendo una velocita media perfetta di 2.5 pixel/frame.
- **Controlli Opposti Simultanei:** Se il giocatore preme contemporaneamente O (Sinistra) e P (Destra), la sezione 4 di Input_CheckCtrl cancella entrambi i bit di direzione preservando solo l'eventuale salto su Z.

3.3 Modello Fisico del Salto Parabolico e Caduta

Il modello balistico di Manic Miner non fa uso di costose librerie a virgola mobile, ma implementa un'elegante gravita frazionaria a precisione intera integrata in Phys_ApplyGravity:

- **Impulso Iniziale:** All'avvio del salto, velocity_y viene caricata con il valore room_jump_strength (default 6). La coordinata Y di partenza viene congelata in jump_start_y.
- **Applicazione Gravita Frazionaria:** Ad ogni frame in aria (in_air), jump_frame viene incrementato. In base al preset attivo (room_gravity_preset = 1, corrispondente a gravita media 0.75), la gravita sottrae 1 da velocity_y per 3 frame su 4 (saltando la sottrazione quando jump_frame & 3 == 3).
- **Spostamento Verticale:** PlayerY viene aggiornato sottraendo velocity_y: PlayerY = PlayerY - velocity_y. Se velocity_y > 0 Willy sale verso l'alto; se velocity_y < 0 Willy scende verso il basso.
- **Velocita Terminale di Caduta:** La velocita di discesa e' limitata superiormente a MAX_FALL_SPEED = 12 pixel per frame per evitare lo sfondamento delle collisioni.

3.4 Limiti di Schermo, Clamp e Coordinate Pixel

L'area di movimento del player e' delimitata da costanti hardware e software:

- **Limite Sinistro (PLAYER_MIN_X = 5):** Se Willy cerca di muoversi oltre il pixel 5, la coordinata viene bloccata a PLAYER_MIN_X e lo sprite non sconfina nel bordo.
- **Limite Destro (PLAYER_MAX_X = 245):** La coordinata X destra non puo superare 245 pixel (permettendo allo sprite largo 16 pixel di fermarsi prima del bordo destro).
- **Limite Verticale Superiore (Y < 0):** Se Willy salta oltre la cima dello schermo (bit 7 di PlayerY = 1), PlayerY viene clampato a 0, velocity_y azzerata e il salto interrotto.
- **Limite Verticale Inferiore (Y >= 113):** Se Willy cade nel baratro sul fondo della caverna (PlayerY >= 113), scatta il controllo di caduta fatale provocando la morte immediata.

3.5 Macchina a Stati del Giocatore e Condizioni di Morte/Respawn

Lo stato del giocatore transita tra condizioni ben definite. La morte (.willy_dies) puo essere causata da:

- **1. Caduta Fatale:** Dislivello verticale (landing_Y - jump_start_y) > 40 pixel (5 blocchi).
- **2. Contatto con Guardiano:** Collisione bit-a-bit pixel-perfect con un pixel attivo di un nemico.
- **3. Contatto con Nasty:** Bounding box di Willy tocca una tile letale fissa (Nasty1 o Nasty2).
- **4. Asfissia:** Esaurimento totale dei 224 pixel della barra dell'aria.

SEQUENZA DI RESPAWN (.NORMAL_RESPAWN)

Alla morte di Willy: 1. Decremento willy_lives; 2. Ripristino di tutti gli item della caverna (Items_ResetCurrentRoomItems); 3. Schermo e bordo a nero con pausa di 0.5s (25 HALT); 4. Azzeramento fisica e impostazione timer sicurezza willy_safety_timer = RESPAWN_SAFETY_TIME; 5. Riposizionamento allo spawn point (OFFSET_POS_WILLY).

4 MOTORE FISICO E RILEVAMENTO DELLE COLLISIONI

4.1 Decompressione del Layout e Mappatura delle Celle (unpacked_layout)

Nel formato originale di Manic Miner, i 512 byte del layout di una stanza (OFFSET_LAYOUT) contengono i codici di attributo grafico delle 16x32 celle. Per garantire prestazioni estreme durante i controlli di collisione ed evitare complesse ricerche lineari a 50Hz, all'ingresso in ogni stanza la routine UnpackRoomLayout analizza ciascun byte e lo converte in un tipo logico standardizzato salvato nel buffer RAM unpacked_layout (512 byte):

ID Tipo	Costante Logica	Tile Originale	Comportamento Fisico
0	Background	OFFSET_BG (544)	Spazio vuoto attraversabile. Nessuna collisione.
1	Floor	OFFSET_FLOOR (553)	Pavimento solido calpestabile. Willy atterra e cammina.
2	Crumbling	OFFSET_CRUMBLING (562)	Pavimento sgretolabile. Solido finche non collassa (8 stadi).
3	Wall	OFFSET_WALL (571)	Muro solido. Blocca movimento orizzontale e salti in testa.
4	Conveyor	OFFSET_CONVEYOR (580)	Nastro trasportatore solido. Trascina forzatamente il player.
5	Nasty 1	OFFSET_NASTY1 (589)	Ostacolo letale fisso (fuoco, punte, acidi). Provoca morte istantanea.
6	Nasty 2	OFFSET_NASTY2 (598)	Ostacolo letale fisso (stalattiti, ragni). Provoca morte istantanea.
7	Extra / Special	OFFSET_EXTRA_TILE (607)	Solido come Floor in alcune caverne (Telefoni, Foresta, Miniera, Banca).

4.2 Rilevamento Collisioni con Pavimenti e Piattaforme (Phys_CheckSolid)

La funzione Phys_CheckSolid (in physics.asm) determina se le coordinate pixel (C = X, B = Y) corrispondono a una cella solida. La procedura calcola: Row = Y / 8 e Col = X / 8, ed interroga unpacked_layout tramite GetCellFromUnpacked:

- **Controllo ai Piedi di Willy:** Per verificare se Willy poggia su una piattaforma, il motore esamina due punti critici a quota Y + 16: il piede sinistro (X, Y + 16) e il piede destro (X + 7, Y + 16). Se almeno uno dei due punti tocca una cella solida (Floor, Crumbling, Wall, Conveyor), Willy e' considerato a terra (on_ground = 1).
- **Tolleranza di Atterraggio da Sopra (.check_landing):** Durante la caduta, per evitare che Willy rimanga bloccato atterrando su pareti verticali laterali o agganci piattaforme arrivando da sotto, l'atterraggio e' consentito solo se la quota precedente dei piedi era entro i primi 3 pixel dalla superficie superiore: VecchioY + 16 <= (Row * 8 + 3). Se Willy proveniva da sotto, attraversa la piattaforma verso l'alto!

4.3 Collisioni con Muri e Soffitti (CheckWillyWallCollision, CheckWallCell)

I muri verticali e i soffitti orizzontali (tipo 3 = Wall) bloccano rigidamente il movimento:

- **Collisione Muro Orizzontale:** La routine CheckWillyWallCollision verifica 3 punti lungo il bordo di avanzamento: vertice superiore (X, Y), centro del corpo (X, Y + 7) e base (X, Y + 15). Se anche un solo punto tocca un Wall, la coordinata X non viene incrementata.
- **Collisione Soffitto (Testa):** Durante la salita in salto (velocity_y > 0), il motore controlla se la testa a sinistra (X, Y) o a destra (X + 7, Y) tocca un Wall. In caso di impatto, PlayerY viene clampato immediatamente sotto il soffitto (riga r + 1) e velocity_y viene azzerata, facendo iniziare la caduta.

4.4 Meccanica Dinamica dei Pavimenti Sgretolabili (Phys_CheckCrumblingTrigger)

I pavimenti sgretolabili costituiscono uno degli enigmi fisici piu celebri del gioco:

- **Griglia crumbling_grid (512 Byte):** Ogni cella della matrice 16x32 possiede un contatore di stadio (da 0 a 8). 0 = blocco intatto; 1..7 = stadi di dissoluzione; 8 = collasso completo.
- **Avanzamento ad Ogni Frame:**Quando Willy poggia i piedi su una tile Crumbling (Phys_CheckCrumblingTrigger), la routine .advance_crumble_cell incrementa lo stadio di 1 e ridisegna la tile (Gfx_DrawCrumblingTile) traslando il pattern bitmap verso il basso di 1 pixel.
- **Collasso Finale:** Non appena lo stadio raggiunge 8, la cella corrispondente in unpacked_layout viene sovrascritta con 0 (Background). Il blocco cessa all'istante di essere solido e Willy precipita nel vuoto.

4.5 Modello Matematico delle Rampe Inclinate (GetRampY)

Le rampe inclinate a 45 gradi consentono a Willy di salire e scendere fluidamente pixel per pixel. La quota teorica della rampa viene calcolata analiticamente dalla funzione GetRampY:

- **Rampa che Sale a Destra (ramp_dir == 1):** Data la colonna di partenza ramp_col e la riga di base ramp_row: ramp_Y = (ramp_row * 8 - 16) - (PlayerX - ramp_col * 8).

- **Rampa che Sale a Sinistra (ramp_dir == 0):** Data la colonna di partenza ramp_col: ramp_Y = (ramp_row * 8 - 16) - (ramp_col * 8 - PlayerX).
- **Aggancio e Guardia d'Ingresso (ramp_entry_guard):** Se Willy entra orizzontalmente in una stanza con una rampa, per evitare che venga teletrasportato sulla rampa dal pavimento sottostante, la guardia disattiva l'aggancio finche Willy non e' uscito dalla zona di confine o non e' gia alla quota corretta.

4.6 Nastri Trasportatori e Logica di Cattura (CheckWillyOnConveyor)

La gestione del nastro trasportatore implementa la celebre dinamica irreversibile del gioco originale:

- **Rilevamento:** CheckWillyOnConveyor verifica se i piedi poggiano su una cella Conveyor (tipo 4) o entro i limiti definiti da conveyor_row, conveyor_col e conveyor_len.
- **Cattura e Trascinamento:** Se Willy cammina nella direzione opposta allo scorrimento del nastro tenendo premuto il tasto, puo avanzare; ma non appena il tasto viene rilasciato, conveyor_caught viene impostato a 1. Da quel momento ogni pressione contraria viene ignorata e Willy viene trascinato verso l'estremita.

4.7 Rilevamento Ostacoli Letali (Phys_CheckNastyCollision)

Phys_CheckNastyCollision protegge Willy controllando 6 punti strategici del suo bounding box (16x8 pixel): Top-Left (X, Y), Top-Right (X+7, Y), Mid-Left (X, Y+7), Mid-Right (X+7, Y+7), Bot-Left (X, Y+15), Bot-Right (X+7, Y+15). Se uno qualsiasi di questi punti tocca una cella Nasty 1 (tipo 5) o Nasty 2 (tipo 6), la funzione ritorna Carry = 1 scatenando la morte di Willy.

5 SISTEMA DEI NEMICI E INTELLIGENZA ARTIFICIALE

5.1 Tabella dei Nemici Attivi (ActiveEnemies) e Struttura dello Slot

I guardiani (nemici mobili) presenti nella caverna corrente vengono istanziati all'ingresso nella tabella ActiveEnemies allocata in RAM. La tabella e' composta da 8 slot da 11 byte ciascuno (88 byte totali):

Offset	Campo Dati	Tipo / Range	Significato e Funzione Logica
+0	Flags	Bitfield	Bit 7: Active (1=attivo), Bit 6: Vertical (1=vert), Bit 5: Eugene/Kong, Bit 4: Slow.
+1	Sprite Frame Index	0..7	Indice del frame grafico (da moltiplicare per 32 per l'offset bitmap).
+2	Current X	0..248 (pixel)	Posizione orizzontale attuale dello sprite 16x16 (Top-Left).
+3	Current Y	0..120 (pixel)	Posizione verticale attuale dello sprite 16x16 (Top-Left).
+4	Min Limit	0..248 (pixel)	Limite minimo del percorso (X min per orizzontali, Y min per verticali).
+5	Max Limit	0..248 (pixel)	Limite massimo del percorso (X max per orizzontali, Y max per verticali).
+6	Direction / Status	1 o 2	1 = Sinistra / Alto, 2 = Destra / Basso (o stato Kong: 0=ledge, 1=falling, 2=dead).
+7	Anim State / Speed	0..7 / 1..2	Frame di animazione (0..7) per orizzontali; step in pixel (1..2) per verticali.
+8	Old X	0..248 ($FF)	Coordinata X del frame precedente, usata per la cancellazione differenziale.
+9	Old Y	0..120 ($FF)	Coordinata Y del frame precedente, usata per la cancellazione differenziale.
+10	Color Attribute	0..127	Attributo colore ZX Spectrum (Bright, Paper, Ink) applicato allo sprite 16x16.

5.2 Guardiani Orizzontali: Ciclo Animazione a 8 Frame e Inversione

I guardiani orizzontali seguono l'algoritmo classico di Manic Miner (routine originale $8D0F):

> **Ciclo dei Frame (0..7):** I frame 0, 1, 2, 3 rappresentano il movimento verso destra all'interno della cella a 8 pixel. I frame 7, 6, 5, 4 rappresentano il movimento verso sinistra.

> **Avanzamento Cella e Limiti:** Quando il frame raggiunge 3 (bordo destro della cella), se Current X < Max Limit X, Current X avanza di 8 pixel e il frame torna a 0. Se Current X ha raggiunto Max Limit X, il guardiano inverte la marcia impostando il frame a 7 e la direzione a 1 (Sinistra). Analogamente, al raggiungimento del frame 4 a sinistra, indietreggia di 8 pixel finche non tocca Min Limit X, dove inverte verso destra.

> **Guardiani Lenti (Slow Speed):** Se il bit 4 dei Flags e' impostato, il guardiano viene aggiornato solo sui frame pari (counter & 1 == 0), dimezzando la sua velocita di pattugliamento.

5.3 Guardiani Verticali: Pattugliamento Lineare e Velocita Frazionaria

I guardiani verticali (Bit 6 impostato) si muovono lungo una colonna fissa:

> **Movimento in Pixel:** Ad ogni frame in cui i nemici vengono aggiornati, la coordinata Y viene incrementata o decrementata dello step (1 o 2 pixel) indicato all'offset +7.

> **Inversione di Marcia:** Al raggiungimento di Min Limit Y (in alto) la direzione passa a 2 (Down); al raggiungimento di Max Limit Y (in basso) la direzione passa a 1 (Up).

> **Animazione:** L'indice del frame grafico (offset +1) cicla continuamente tra 0, 1, 2 e 3.

5.4 Comportamento Speciale del Boss Eugene (Caverna 04)

Eugene, il boss della Caverna 04 (Eugene's Lair), e' allocato nello Slot 4 di ActiveEnemies con flag $E0 (Active, Vertical, Eugene):

> **Fase 1: Pattugliamento:** Finché restano oggetti da raccogliere (game_objective_completed == 0), Eugene pattuglia verticalmente tra Y=0 e Y=88 con colore bianco su rosso (Paper 2, Ink 7 = 23).

> **Fase 2: Discesa sul Portale e Arcobaleno:** Non appena tutti gli oggetti sono stati raccolti, Eugene scende verso la base (Y=88) bloccando l'accesso al portale e inizia a cambiare colore a ogni frame con un effetto arcobaleno sincronizzato con il timer interrupt (counter).

5.5 Meccaniche del Boss Kong Beast e Interruttori (Caverne 07 e 11)

Nelle caverne 'Miner Willy meets the Kong Beast' e 'Return of the Alien Kong Beast', Kong Beast e' posizionato su un cornicione in alto a destra (Y=0, X=120, Slot 3 di ActiveEnemies). La sconfitta di Kong richiede l'azionamento sequenziale di due interruttori speciali (Kong_CheckSwitches):

> **1. Interruttore Sinistro (Row 0, Col 6):** Quando Willy tocca l'interruttore sinistro (kong_switch_left = 1), l'interruttore scatta graficamente a video, il muro alle celle (Row 11-12, Col 17) viene abbattuto in unpacked_layout e il percorso del guardiano 2 viene esteso fino a Col 28.

> **2. Interruttore Destro (Row 0, Col 18):** Quando Willy tocca l'interruttore destro (kong_switch_right = 1), l'interruttore scatta, il pavimento sotto Kong a (Row 2, Col 15-16) crolla, Kong precipita verso il basso (Y aumenta di 2 px/frame con frame 2-3) e all'impatto a Y=88 viene rimosso assegnando 100 punti bonus al giocatore.

5.6 Collision Detection Pixel-Perfect Bit-a-Bit (CheckCollision)

Manic Miner e' universalmente celebre per la sua collision detection rigorosamente bit-a-bit (CheckCollision in enemy.asm):

> **Fase 1: Pre-filtro Bounding Box Rapido:** Calcola DeltaX = WillyX - EnemyX (valido se tra -7 e +15) e DeltaY = WillyY - EnemyY (valido se tra -15 e +15). Se fuori range, il nemico viene scartato all'istante a costo quasi zero.

> **Fase 2: Calcolo Righe Sovrapposte:** Determina overlap_num_lines (numero di scanline orizzontali in cui Willy e il guardiano si intersecano, da 1 a 16).

> **Fase 3: Allineamento Shift a 16 Bit:** Per ciascuna scanline sovrapposta, i 2 byte grafici del guardiano vengono estratti e traslati a sinistra o a destra con istruzioni sla e / rl d per allinearli esattamente alla coordinata pixel di Willy.

> **Fase 4: Operazione AND Logica:** Il byte allineato del guardiano viene confrontato con il byte della scanline corrispondente di Willy (testa o corpo) tramite l'istruzione AND C. Se il risultato e' diverso da zero (AND C != 0), c'e un contatto fisico reale tra pixel opachi -> CARRY SET (Collisione Confermata!).

6 OGGETTI, PUNTEGGIO E MECCANICHE DI GAMEPLAY

6.1 Struttura e Caricamento degli Oggetti della Caverna

Gli oggetti collezionabili (chiavi e tesori) sono definiti all'interno della struttura della caverna a partire dall'offset OFFSET_ITEMS (629). Ciascuna stanza puo contenere fino a 5 oggetti. Ogni record oggetto occupa 5 byte:

Byte	Campo	Descrizione	Esempio Valore
+0	Color Attribute	Attributo colore iniziale ZX Spectrum (Paper, Ink, Bright)	3 (Magenta su nero)
+1	Attr Address Low	Byte basso dell'indirizzo video attributi (23552 + Row*32 + Col)	$89 (per Col 9, Row 0)
+2	Attr Address High	Byte alto dell'indirizzo video attributi ($58..$5A)	$5C (23552 + offset)
+3	Anim Mask	Maschera per la rotazione cromatica ciclica	96 ($60)
+4	Collected Flag	$FF = attivo / non raccolto, 0 = raccolto	$FF (Iniziale)

All'avvio della stanza, la funzione Items_DrawRoom copia gli oggetti attivi nella cache veloce current_room_items (5 byte per slot: Col, Row, Color, FlagPtrLow, FlagPtrHigh) e popola la matrice di lookup rapido items_grid (512 byte in RAM). La bitmap grafica comune a tutti gli item della stanza viene letta dagli 8 byte situati all'offset OFFSET_ITEM_GRAPHIC (692).

6.2 Animazione Ciclica del Colore degli Item (Items_Animate)

Gli oggetti di Manic Miner non possiedono animazione bitmap a piu frame, ma si animano tramite la celebre rotazione degli attributi colore INK a 50Hz (routine originale $8F63):

- **Rotazione Colori Ink:** Ad ogni frame, la routine Items_Animate cicla l'inchiostro attraverso la sequenza: Magenta (3) -> Verde (4) -> Ciano (5) -> Giallo (6) -> Magenta (3), preservando inalterati i bit di PAPER e BRIGHT.
- **Aggiornamento Diretto VRAM:** Il nuovo attributo cromatico viene scritto simultaneamente sia nella cache current_room_items, sia nella tabella items_grid, sia direttamente nella memoria video degli attributi VRAM ($5800 + Row*32 + Col), creando il caratteristico scintillio.

6.3 Raccolta Oggetti e Sistema di Scoring BCD a 6 Cifre

Ad ogni frame, Items_CheckCollision esegue un test di intersezione bounding box (16x16 pixel di Willy contro 8x8 pixel dell'oggetto):

- **Riconoscimento Raccolta:** Se Willy tocca un oggetto attivo: 1. Il byte +4 nei dati stanza viene azzerato; 2. La cella viene rimossa da items_grid; 3. La cella di sfondo viene ripristinata graficamente tramite RestoreBackgroundCell; 4. Viene emesso il beep audio di raccolta (Audio_PlayCollectSound); 5. Viene decrementato il contatore num_room_items; 6. Vengono assegnati 100 punti.
- **Aritmetica BCD con Istruzione DAA:** Il punteggio player_score e' memorizzato in 3 byte BCD (6 cifre decimali). La routine Score_AddItemPoints incrementa il byte 1 (centinaia/migliaia) ed esegue l'istruzione hardware Z80 DAA (Decimal Adjust Accumulator) con propagazione del riporto al byte 2. Questo elimina la necessita di costose routine di conversione binario-decimale a runtime!

6.4 Gestione dell'High Score Persistente

L'High Score (high_score, 3 byte BCD) viene preservato in memoria tra le varie partite:

- **Confronto Big-Endian:** La funzione Score_CheckAndUpdateHighScore confronta i 3 byte BCD partendo dalla cifra piu significativa (byte 2: centinaia di migliaia) fino alla meno significativa (byte 0). Se player_score supera high_score, il valore viene copiato istantaneamente e il display HUD aggiornato.

6.5 Portale di Fine Livello e Svuotamento Riserva d'Aria (Air_DepleteCountdown)

Quando tutti gli oggetti della stanza sono stati raccolti (num_room_items == 0, game_objective_completed = 1), il Portale d'uscita (16x16 pixel) inizia a lampeggiare (Gfx_AnimatePortal).

- **Countdown dell'Aria (Air_DepleteCountdown):** Quando Willy entra nel portale, il gioco congela la fisica ed esegue un ciclo ad alta velocita che svuota la barra dell'aria 1 pixel alla volta: per ciascun pixel consumato, viene aggiunto 1 punto allo score del giocatore (Score_Add1Point) e viene emesso un impulso audio sul buzzer con frequenza inversamente proporzionale all'aria residua.

6.6 Sequenza di Game Over (Piede) e Schermata di Vittoria (Maria)

- **Game Over (TriggerGameOver):** All'esaurimento delle vite, il gioco lancia l'animazione originale del Piede gigante (Monty Python) che scende progressivamente dall'alto schiacciando Willy sul piedistallo, accompagnata da note del beeper e dalla scritta 'Game Over' con effetto cromatico arcobaleno (glistening).
- **Vittoria Finale (TriggerVictory):** Al superamento dell'ultima caverna, viene visualizzata la schermata di vittoria con Maria (lo sprite della 'Vecchia') che cammina verso Willy, la scritta lampeggiante 'CONGRATULATIONS!' e il prompt interattivo 'PLAY AGAIN? Y/N'.

7 STRUTTURA DATI DELLE CAVERNE (1024 BYTE STANDARD)

7.1 Mappa Completa dei 1024 Byte della Caverna

Ogni livello del gioco Manic Miner e' memorizzato in un blocco binario compatto e rigidamente vincolato a 1024 byte (1 KB). Questa tabella autocontenuta racchiude l'intera definizione geometrica, cromatica, fisica ed entitaria della caverna:

Offset	Dim.	Etichetta Simbolica	Descrizione del Blocco Dati
0..511	512 B	CavernLayout	Matrice 16x32 delle celle di gioco (codici di attributo grafico delle tile).
512..543	32 B	CavernTile / RoomName	Nome della caverna in formato stringa ASCII (esattamente 32 caratteri).
544..615	72 B	CavernTilesDef (8x9)	8 definizioni di tile (1 byte attr + 8 byte bitmap: BG, Floor, Crumble, Wall, Conv, Nasty1, Nasty2, Extra).
616..622	7 B	WillyAppearanceAndPos	Posizione iniziale: Byte 0: Y*2, Byte 1: frame, Byte 2: dir, Byte 4-5: AttrAddr.
623..626	4 B	ConveyorData	Nastro trasportatore: Byte 0: dir (0=SX, 1=DX), Byte 1-2: ScreenAddr, Byte 3: len.
627	1 B	BorderColor	Colore del bordo video dello ZX Spectrum per la caverna (valore 0..7).
628	1 B	Unused1	Byte di riserva non utilizzato (impostato a 0).
629..654	26 B	LocationColorItems	Fino a 5 oggetti (5 byte cad.: Attr, AttrAddr Low/High, Anim, Flag) + terminatore $FF.
655..691	37 B	PortalData	Portale 16x16: Attributo, 32 byte bitmap (4 frame da 8 byte), AttrAddr Low/High.
692..699	8 B	ItemGraphic	Bitmap 8x8 comune per tutti gli oggetti collezionabili della caverna.
700	1 B	InitialAir	Riserva d'aria iniziale disponibile nella caverna (es. 63).
701	1 B	GameClock	Velocita dell'orologio / timer per il consumo d'aria della stanza.
702..730	29 B	HorizontalGuardians	Fino a 4 guardiani orizzontali (7 byte cad.: Attr, AttrAddr, Mask, Frame, MinX, MaxX) + $FF.
731..732	2 B	Unused2	Byte non utilizzati (impostati a 0, 0).
733	1 B	VGuardians	Guardiani verticali (o terminatore $FF; se 0 indica stanza con boss).
734..735	2 B	Unused3	Byte non utilizzati (impostati a 0, 0).
736..767	32 B	GraphicElement	Grafica speciale per boss (Eugene, Kong) o elementi paesaggistici speciali.
768..1023	256 B	GraphicGuardian	Set di sprite dei guardiani (8 frame 16x16 pixel da 32 byte ciascuno).

7.2 Caricamento, Inizializzazione e Transizione tra Stanze

All'ingresso in una caverna, la routine LoadRoomAndRestart (in manicminer.asm) esegue la seguente pipeline atomica:

> **1. Reset Stack e Stato:** Ripristina lo stack pointer (ld sp, $fdf0) per azzerare qualsiasi residuo di chiamate precedenti; azzera la griglia dei pavimenti sgretolabili (crumbling_grid) e i timer dei portali.

> **2. Decompressione e Disegno:** Esegue UnpackRoomLayout; chiama Gfx_DrawRoom per disegnare l'intera matrice 16x32; disegna conveyor e rampe (Gfx_DrawConveyorsAndRamps); stampa il nome della caverna a riga 16 e inizializza la barra dell'aria a riga 17.

> **3. Inizializzazione Entita:** Chiama Items_DrawRoom per disegnare e registrare gli oggetti; chiama Enemy_LoadRoom per inizializzare ActiveEnemies con guardiani orizzontali, verticali o boss.

> **4. Posizionamento Willy e Bordo:** Legge le coordinate di spawn da OFFSET_POS_WILLY (se transition_active == 0); legge il colore del bordo da OFFSET_BORDER_COLOR, lo invia alla porta hardware out ($fe), a e aggiorna la variabile BORDCR della ROM; disegna Willy.

7.3 Analisi Dettagliata delle 20 Caverne del Gioco

Tabella sinottica delle 20 caverne originali incluse nel motore:

ID	Nome Caverna	Colore Bordo	Caratteristiche, Guardiani e Meccaniche Speciali
00	Central Cavern	Rosso (2)	Livello introduttivo; 5 chiavi, 1 conveyor a sinistra, guardiani minerari orizzontali.
01	The Cold Room	Blu (1)	Pavimenti ghiacciati; guardiani pinguini orizzontali e blocchi sgretolabili.
02	The Menagerie	Magenta (3)	Presenza di molteplici animali e mostri guardiani; percorsi multilivello.
03	Abandoned Uranium Workings	Verde (4)	Ostacoli radioattivi Nasty; salti di precisione millimetrica su blocchi singoli.
04	Eugene's Lair	Rosso (2)	Boss Eugene al centro; scende sul portale e diventa arcobaleno alla raccolta degli item.
05	Processing Plant	Ciano (5)	Nastri trasportatori multipli contrapposti e nastri sgretolabili combinati.

ID	Nome Caverna	Colore Bordo	Caratteristiche, Guardiani e Meccaniche Speciali
06	The Vat	Giallo (6)	Vasca profonda; richiede una discesa controllata senza superare l'altezza di caduta sicura.
07	Miner Willy meets the Kong Beast	Verde (4)	Boss Kong Beast in alto a destra; 2 interruttori per aprire il varco e far precipitare il boss.
08	Wacky Amoebatrons	Blu (1)	Guardiani amebe con percorsi orizzontali sovrapposti e piattaforme sgretolabili.
09	The Endorian Forest	Verde (4)	Tile Extra (7) solido come Floor; alberi e rami percorribili, guardiani ewok.
10	Attack of the Mutant Telephones	Magenta (3)	Tile Extra solido; guardiani telefoni giganti; nastri trasportatori veloci.
11	Return of the Alien Kong Beast	Giallo (6)	Seconda battaglia contro Kong; interruttori e guardiani alieni sincronizzati.
12	Ore Refinery	Rosso (2)	Raffineria con pavimenti metallici e acido; tile Extra solido come Floor.
13	Skylab Landing Bay	Ciano (5)	Guardiani Skylab verticali che precipitano dall'alto simulando detriti spaziali.
14	The Bank	Verde (4)	Caveau della banca; tile Extra solido; casseforti e guardiani custodi.
15	The Sixteenth Cavern	Blu (1)	Labirinto verticale complesso con piattaforme sgretolabili a cascata.
16	The Warehouse	Giallo (6)	Magazzino con barili e casse rotolanti; salti con raggio d'azione strettissimo.
17	Amoebatrons' Revenge	Magenta (3)	Ritorno delle amebe mutanti; densita di ostacoli Nasty elevatissima.
18	Solar Power Generator	Giallo (6)	Generatore solare; raggi laser letali e piattaforme mobili a tempo.
19	The Final Barrier	Bianco (7)	Livello finale; paesaggio montano esterno (datalandscape.asm); porta alla Master Bedroom!

8 GRAFICA, VRAM E RENDERING FLICKER-FREE

8.1 Organizzazione e Geometria della VRAM dello ZX Spectrum ($4000..$5AFF)

L'architettura grafica dello ZX Spectrum presenta una mappa di memoria storicamente non lineare, concepita dalla Sinclair per ottimizzare il circuito integrato ULA. La memoria video e' divisa in due blocchi contigui:

> **1. Display Pixel RAM ($4000..$57FF, 6144 Byte):** Risoluzione 256x192 pixel monocromatici a 1 bit per pixel (1 byte = 8 pixel orizzontali). L'area e' suddivisa in 3 terzi verticali da 64 scanline ciascuno. All'interno di ciascun terzo, le scanline non sono consecutive ma interlacciate a blocchi di 8 righe di caratteri.

> **2. Attribute RAM ($5800..$5AFF, 768 Byte):** Matrice di 32 colonne x 24 righe di caratteri (8x8 pixel). Ciascun byte definisce i colori della cella corrispondente secondo il layout di bit standard: Bit 7: FLASH (lampeggio), Bit 6: BRIGHT (luminosita), Bit 5-3: PAPER (colore sfondo 0..7), Bit 2-0: INK (colore tracciato 0..7).

8.2 Calcolo Matematico degli Indirizzi Video e Scanline (Gfx_GetScreenAddr)

Per convertire una coordinata cartesiana (L = X in pixel 0..255, H = Y in scanline 0..191) nell'indirizzo fisico a 16 bit della VRAM, la routine Gfx_GetScreenAddr applica la seguente manipolazione di bit:

graphics.asm - Gfx_GetScreenAddr

```
; Calcolo indirizzo VRAM da coordinate cartesiane (H=Y, L=X)
; Formato Byte Alto (D): 0 1 0 y7 y6 y2 y1 y0
; Formato Byte Basso (E): y5 y4 y3 x4 x3 x2 x1 x0
Gfx_GetScreenAddr:
    ld   a, h
    and  %00000111         ; Isola i bit di scanline y2 y1 y0
    or   %01000000         ; Base memoria video $40 (0100 0000)
    ld   d, a              ; D parziale
    ld   a, h
    and  %11000000         ; Isola i bit del terzo y7 y6
    rrca : rrca : rrca     ; Shift a destra di 3 posizioni
    or   d
    ld   d, a              ; D completo
    ld   a, h
    and  %00111000         ; Isola la riga del carattere y5 y4 y3
    add  a, a : add a, a   ; Shift a sinistra di 2 posizioni
    ld   e, a              ; E parziale
    ld   a, l
    srl a : srl a : srl a  ; A = X / 8 (colonna orizzontale 0..31)
    or   e
    ld   e, a              ; E completo
    ld   h, d : ld l, e    ; HL = Indirizzo fisico esatto in VRAM
    ret
```

8.3 Rendering degli Sprite e Ricostruzione dello Sfondo (Gfx_GetBackgroundByte)

Poiche lo ZX Spectrum 48K non possiede memoria sufficiente per un secondo buffer video da 6.75 KB (double buffering), Manic Miner adotta una sofisticata pipeline di rendering differenziale a cancellazione selettiva:

> **Cancellazione Intelligente senza Sfarfallio:** Quando un guardiano o Willy si sposta, la routine Gfx_ErasePixelSprite16x16 non cancella lo sprite disegnando nero, ma interroga Gfx_GetBackgroundByte. Questa funzione calcola analiticamente quale elemento grafico si trova in quella posizione (matrice tile, nastro trasportatore, pavimento sgretolabile o portale) e riscrive i byte originali della scena!

> **Ripristino Attributi Colore:** Contemporaneamente, Gfx_RestoreEnemyColorAttributes ripristina gli esatti attributi cromatici dello sfondo presenti prima del passaggio del nemico, prevenendo l'attribute bleeding.

8.4 Pipeline di Rendering a Due Passate e Risoluzione Sovrapposizioni

Per eliminare qualsiasi sfarfallio e garantire che Willy rimanga sempre visibile in primo piano rispetto ai guardiani, il ciclo di rendering si suddivide in due passate atomiche:

> **Passata 1 (Erase Pass):** Tutti i nemici attivi vengono cancellati nella loro vecchia posizione (Old X, Old Y). Durante questa fase, la funzione CheckEnemyWillyOverlap controlla se la vecchia posizione del nemico intersecava Willy: in caso affermativo, imposta il flag willy_needs_redraw = 1.

> **Passata 2 (Draw Pass):** Tutti i nemici vengono ridisegnati nella nuova posizione (Current X, Current Y). CheckEnemyWillyOverlap verifica nuovamente se la nuova posizione si sovrappone a Willy: in tal caso, reimposta willy_needs_redraw = 1.

> **Ridisegno Willy in Primo Piano (Gfx_UpdatePlayer):** Subito dopo il disegno dei nemici, Gfx_UpdatePlayer verifica se Willy si e' mosso oppure se willy_needs_redraw == 1: in entrambi i casi cancella e ridisegna immediatamente Willy, assicurando che il protagonista compaia SEMPRE sopra qualsiasi guardiano!

8.5 HUD, Barra dell'Aria Dinamica e Animazione delle Vite

L'interfaccia a video occupa le righe da 16 a 23 dello schermo:

- **Nome Stanza (Riga 16):** Centrato su 32 colonne, con colori vividi associati alla caverna.
- **Barra dell'Aria (Riga 17):** 28 colonne di pixel (224 pixel totali). Gfx_UpdateAirBar aggiorna con precisione di 1 singolo pixel l'estremita della barra verde/rossa senza ridisegnare l'intero schermo.
- **Riga di Stato e Vite (Righe 18..23):** Visualizza 'High Score: XXXXXX', 'Score: XXXXXX' e le icone animate di Willy che camminano sul fondo dello schermo indicando le vite rimanenti.

9 GAME LOOP, SINCRONIZZAZIONE E GESTIONE INPUT

9.1 Ciclo Principale di Gioco (Main_loop) e Sequenza Rigorosa delle Fasi

Il ciclo principale di gioco (Main_loop in manicminer.asm) orchestra l'intero funzionamento del programma con una sequenza temporale rigorosa e deterministica a 25 frame al secondo:

Fase	Modulo / Funzione Chiamata	Operazioni Eseguite e Obiettivo
1	.sync_loop (HALT)	Attende il vertical blank hardware della ULA a 50Hz. Modula la velocita tramite GAME_SPEED_DELAY (2 interrupt = 25 FPS).
2	willy_safety_timer	Decrementa il timer di sicurezza (invulnerabilita post-respawn) se > 0.
3	Enemy_RenderRoom	Passata atomica: cancella vecchi nemici, controlla sovrapposizioni con Willy (willy_needs_redraw) e disegna nuovi nemici.
4	Gfx_UpdatePlayer	Cancella e ridisegna Willy in primo piano se mosso o se coperto da un guardiano.
5	Gfx_AnimatePortal	Se tutti gli item sono raccolti, fa lampeggiare ciclicamente l'attributo del portale.
6	Phys_UpdateAir	Decrementa la riserva d'aria in base ad air_deplete_rate; se l'aria e' a 0, salta a .willy_dies.
7	Audio_UpdateJumpSound	Esegue l'effetto sonoro del salto frame-by-frame se attivo (suono modulato a 18 note).
8	Audio_UpdateMusic	Aggiorna la musica di sottofondo polifonica a 1-bit se abilitata.
9	Gfx_PrintWillyLives	Aggiorna l'animazione camminante delle icone delle vite in basso a schermo.
10	Salvataggio PlayerPosOld	Copia PlayerPos in PlayerPosOld per preparare la cancellazione al frame successivo.
11	Input_CheckCtrl	Scansiona le porte I/O della tastiera ($DF, $FE, $FB, $FD), popola registro D e applica i filtri.
12	MovePlayer	Calcola lo spostamento orizzontale di Willy, gestisce collisioni muri e cattura nastro.
13	Phys_ApplyGravity	Applica la gravita frazionaria, salto, atterraggio, rampe e crumbling floors; controlla caduta fatale.
14	Enemy_Update	Aggiorna le posizioni e i frame dei guardiani orizzontali, verticali, Eugene e Kong.
15	Items_CheckCollision	Verifica raccolta chiavi, assegna 100 punti BCD, emette beep e anima i colori degli item.
16	CheckPortalTransition	Verifica se Willy entra nel portale attivo -> Scatena Air_DepleteCountdown e cambio caverna.

9.2 Sincronizzazione Hardware Vertical Blank via Interrupt IM2 a 50Hz

Per garantire un frame rate costante e privo di tearing video, il gioco sfrutta la modalita di interrupt vettorizzato IM 2 dello Z80:

- **Inizializzazione IM2 (SetupIM2):** Carica il registro I con $FE; riempie la tabella da 257 byte ($FE00..$FEFF) con il byte $FD; scrive all'indirizzo $FDFD un'istruzione di salto assoluto jp im2_handler (opcode $C3 seguito dall'indirizzo dell'handler); attiva IM 2 ed abilita gli interrupt con EI.
- **Handler Interrupt (im2_handler):** Eseguito esattamente ogni 20 millisecondi (50 volte al secondo) dal segnale di sincronismo verticale della ULA. Incrementa la variabile a 16 bit counter, legge la porta $FE per resettare la linea di interrupt e ritorna con RETI.

9.3 Scansione della Tastiera, Matrice delle Porte I/O Spectrum

L'input viene letto direttamente scansionando le linee della matrice di commutazione tastiera dello Spectrum (routine Input_CheckCtrl in input.asm):

Porta Hardware	Linee / Bit	Tasti Associati	Funzione nel Gioco
IN A, ($DF)	Bit 0 (P), Bit 1 (O)	P = Destra, O = Sinistra	Movimento orizzontale di Miner Willy a terra e in salto.
IN A, ($FE)	Bit 1 (Z), Bit 0 (Space)	Z / Space = Salto	Attivazione salto parabolico (imposta bit 2 del registro D).
IN A, ($FB)	Bit 0 (Q), Bit 1 (W)	Q = Stanza Avanti, W = Warp	Cambio stanza immediato (debug/cheat) e selezione numerica caverna.
IN A, ($FD)	Bit 0 (A)	A = Stanza Indietro	Cambio stanza retroattivo alla caverna precedente.
IN A, ($BF)	Bit 0 (ENTER)	ENTER = Avvia / Conferma	Avvio partita da Title Screen e conferma selezioni.

9.4 Coyote Time, Jump Buffer e Gestione Debounce per Controlli Fluidi

Per eliminare la rigidita frustrante tipica dei giochi a 8 bit e garantire un'esperienza di controllo moderna e reattiva, sono state introdotte tre tecniche evolute:

- **1. Jump Buffer (JUMP_BUFFER_FRAMES = 5):** Se il giocatore preme il tasto di salto (Z) fino a 5 frame prima di toccare terra, jump_pressed conserva l'input ed esegue il salto istantaneamente nel momento esatto dell'atterraggio, eliminando i tasti 'sordi'.
- **2. Coyote Time (COYOTE_TIME_FRAMES = 4):** Quando Willy cammina oltre il bordo di una piattaforma cadendo nel vuoto, coyote_timer gli concede una finestra di tolleranza di 4 frame (80ms) durante la quale puo ancora spiccare il salto come se si trovasse a terra.
- **3. Debounce Hardware (jump_prev, q_prev, a_prev):** I tasti speciali e il salto implementano edge detection: l'azione scatta solo sul fronte di salita della pressione e non si ripete finché il tasto non viene rilasciato.

10 OTTIMIZZAZIONI Z80 E TECNICHE A BASSO LIVELLO

10.1 Tecniche di Risparmio di Memoria e Compattazione Dati

Sviluppare un gioco ricco di 20 livelli, decine di sprite animati, audio polifonico e fisica accurata nei 48 Kilobyte di RAM dello ZX Spectrum ha imposto strategie di compattazione estreme:

- **Formato Monolitico a 1024 Byte per Livello:** Ogni stanza e' un blocco binario rigorosamente allineato a 1024 byte (1 KB). Tutte le 20 caverne occupano esattamente 20480 byte (dal $E000 al $F7FF). I dati di layout (512 byte) utilizzano codici di attributo diretti, condividendo la grafica delle 8 tile definite all'interno della stanza stessa.
- **Sprite Condivisi e Mascherati:** Tutti i guardiani orizzontali di ciascuna caverna condividono lo stesso set di 8 frame da 32 byte situato a OFFSET_GUARDIAN_GRAPHICS (256 byte totali). I guardiani verticali utilizzano i frame 0..3 della stessa tabella, eliminando duplicazioni di memoria.
- **Decompressione Dinamica del Layout (unpacked_layout):** Invece di memorizzare matrici di collisione separate per ogni caverna, la singola matrice di 512 byte viene generata dinamicamente in RAM all'ingresso nella stanza analizzando le 8 definizioni di tile, risparmiando oltre 10 KB di ROM.

10.2 Ottimizzazioni di Velocita nei Calcoli Critici a 16-Bit

Il microprocessore Z80 non dispone di istruzioni hardware di moltiplicazione o divisione. I calcoli matematici critici a 50Hz sono stati ottimizzati tramite shift veloci e aritmetica a 16 bit:

- **Moltiplicazione per 32 (Row * 32):** Eseguita in soli 5 cicli di add hl, hl (HL = Row; 5 raddoppi = moltiplicazione per 32 in appena 55 T-States, contro le centinaia di cicli di una routine generica).
- **Shift Rapido a 16 Bit per Allineamento Pixel:** Nella routine di collision detection (CheckCollision), lo scorrimento dei 16 pixel del guardiano viene eseguito accoppiando i registri D ed E con la sequenza sla e / rl d: un solo shift combinato ad alta efficienza.
- **Uso dei Registri Alternativi (EXX):** Nelle routine di stampa testo e posizionamento cursore (Gfx_At, Gfx_Ink), lo scambio rapido dei banchi di registri con l'istruzione EXX consente di preservare i registri primari (BC, DE, HL) senza dover accedere allo stack con lente istruzioni PUSH/POP.

10.3 Gestione Atomica della VRAM per Eliminazione Sfarfallio

L'architettura hardware dello Spectrum rende impossibile il double-buffering tradizionale per mancanza di RAM video secondaria. Manic Miner risolve questo limite storico con una tecnica proprietaria di rendering atomico:

SCHEMA DELLA PIPELINE DI RENDERING FLICKER-FREE

```
+-------------------------------------------------------------------------+
|              PIPELINE DI RENDERING DIFFERENZIALE FLICKER-FREE           |
+-------------------------------------------------------------------------+
| 1. FASE ERASE:                                                          |
|    Per ogni guardiano attivo:                                           |
|    - Leggi Old X, Old Y.                                                |
|    - Calcola i byte dello sfondo originale tramite Gfx_GetBackgroundByte |
|    - Riscrivi i byte esatti dello sfondo nella VRAM pixel ($4000..$57FF) |
|    - Ripristina gli attributi cromatici originali in VRAM ($5800..$5AFF) |
|    - Controlla se toccava Willy -> willy_needs_redraw = 1               |
|                                                                         |
| 2. FASE DRAW:                                                           |
|    Per ogni guardiano attivo:                                           |
|    - Disegna lo sprite 16x16 a Current X, Current Y                     |
|    - Scrivi gli attributi cromatici del nemico                          |
|    - Controlla se tocca Willy -> willy_needs_redraw = 1                 |
|                                                                         |
| 3. FASE PLAYER OVERLAY:                                                 |
|    Se Willy si e' mosso OPPURE willy_needs_redraw == 1:                 |
|    - Cancella Willy alla vecchia posizione (ripristina sfondo)           |
|    - Ridisegna Willy in primo piano sopra qualsiasi nemico              |
+-------------------------------------------------------------------------+
```

10.4 Aritmetica BCD Hardware e Lookup Table

Per gestire il punteggio a 6 cifre e l'High Score in tempo reale ad ogni frame senza degradare il framerate, il motore adotta l'aritmetica BCD (Binary Coded Decimal):

- **Istruzione DAA (Decimal Adjust Accumulator):** L'istruzione hardware Z80 DAA corregge automaticamente l'accumulatore dopo un'addizione binaria per mantenerlo in formato decimale compresso (due cifre per byte). Questo consente di aggiornare centinaia e migliaia in sole 4 istruzioni macchina (add a, 1; daa; ld (score+1), a; adc a, 0; daa; ld (score+2), a).
- **Lookup Table Precalcolate:** L'indirizzamento delle stanze (RoomsListTable), la mappa delle definizioni tile e le tabelle di ritardo delle note audio sono interamente precompilate come tabelle di puntatori diretti.

11 PORZIONI DI CODICE SIGNIFICATIVE COMMENTATE

11.1 Il Game Loop Principale (manicminer.asm: Main_loop)

Il ciclo principale di gioco sincronizzato a 25 FPS che scandisce tutte le fasi operative:

manicminer.asm - Main_loop

```
Main_loop:
.sync_loop:
    halt                ; Attende interrupt VBLANK della ULA a 50Hz
    ld   hl, (counter)     ; HL = contatore interrupt a 16 bit
    ld   de, (last_frame_counter)
    or   a
    sbc  hl, de            ; Calcola quanti tick sono trascorsi
    ld   a, l
    cp   GAME_SPEED_DELAY  ; 2 tick (50Hz / 2 = 25 FPS)
    jr   c, .sync_loop     ; Se non sono ancora passati 2 tick, attendi
    ld   hl, (counter)
    ld   (last_frame_counter), hl

    ; 1. Rendering atomico nemici ed estrazione sovrapposizioni
    call Enemy_RenderRoom
    ; 2. Rendering del giocatore (overlay in primo piano)
    call Gfx_UpdatePlayer
    ; 3. Aggiorna animazione lampeggio portale
    call Gfx_AnimatePortal
    ; 4. Aggiorna consumo aria (morte se esaurita)
    call Phys_UpdateAir
    jp   c, .willy_dies
    ; 5. Aggiorna audio salto e musica
    call Audio_UpdateJumpSound
    ; 6. Aggiorna fisica, controlli, gravita e collisioni
    call Input_CheckCtrl
    call MovePlayer
    call Phys_ApplyGravity
    jp   c, .willy_dies    ; Caduta fatale
    call Enemy_Update
    call Items_CheckCollision
    call Items_Animate
    call CheckCollision    ; Collisione pixel-perfect con guardiani
    jp   c, .willy_dies
    jp   Main_loop
```

11.2 Movimento del Giocatore e Cattura del Conveyor (player.asm: MovePlayer)

Gestione del movimento a terra, in aria e logica di cattura forzata sui nastri trasportatori:

player.asm - MovePlayer

```
MovePlayer:
    ld   a, (on_ground)
    or   a
    jr   nz, .on_ground_check
    ; IN ARIA: Direzione vincolata a jump_dir, input ignorato
    xor  a : ld (conveyor_caught), a
    ld   a, (jump_dir)
    or   a : ret z         ; Salto verticale -> nessun movimento X
    ld   bc, (PlayerPos)
    cp   1 : jp z, movePlayer_left
    cp   2 : jp z, movePlayer_right
    ret

.on_ground_check:
    call CheckWillyOnConveyor
    jr   c, .on_conveyor_ground
    ; Terreno normale: leggi input D
    xor  a : ld (conveyor_caught), a
    bit  $01, d : jp nz, movePlayer_right
    bit  $00, d : ret z
    jp   movePlayer_left

.on_conveyor_ground:
    ; NASTRO TRASPORTATORE: se rilascia il tasto opposto -> catturato!
    ld   a, (conveyor_dir)
    or   a : jr nz, .conv_is_right
    bit  $01, d            ; Nastro a SX: premuto Destra?
    jr   nz, .check_oppose_right
    ld   a, 1 : ld (conveyor_caught), a ; Catturato!
    jp   movePlayer_left   ; Trascinato a Sinistra
```

11.3 Collision Detection Pixel-Perfect Bit-a-Bit (enemy.asm: CheckCollision)

L'algoritmo di confronto delle scanline sovrapposte tra Willy e i guardiani:

enemy.asm - CheckCollision

```
.scanline_loop:
    push bc
    ; 1. Leggi i 2 byte della scanline corrente del guardiano
    ld   a, (row_enemy_cur) : add a, a : ld e, a : ld d, 0
    ld   hl, (guard_sprite_ptr) : add hl, de
    ld   d, (hl) : inc hl : ld e, (hl) ; D=Byte0, E=Byte1
    ; 2. Allinea a 16 bit la fetta di pixel in base a DeltaX
    ld   a, (collision_delta_x)
    bit  7, a : jr nz, .shift_neg_dx
    or   a : jr z, .slice_ready_in_d
    ld   c, a
.shift_left_16bit:
    sla  e : rl   d        ; Shift combinato a 16 bit!
    dec  c : jr nz, .shift_left_16bit
.slice_ready_in_d:
    ld   a, d              ; A = Fetta guardiano allineata a Willy
    ; 3. Leggi il byte corrispondente dello sprite di Willy
    ld   c, a
    ld   a, (row_willy_cur) ; Leggi testa (0..7) o corpo (8..15)
    ... (lettura byte Willy in A) ...
    ; 4. TEST BIT A BIT: AND Logico tra pixel Willy e pixel Nemico!
    and  c
    jr   nz, .pixel_collision_detected ; Se != 0 -> COLLISIONE FISICA!
    pop  bc : dec b : jp nz, .scanline_loop
```

11.4 Aritmetica BCD per lo Scoring (items.asm: Score_AddItemPoints)

Incremento decimale a 6 cifre senza costose conversioni binario-decimale:

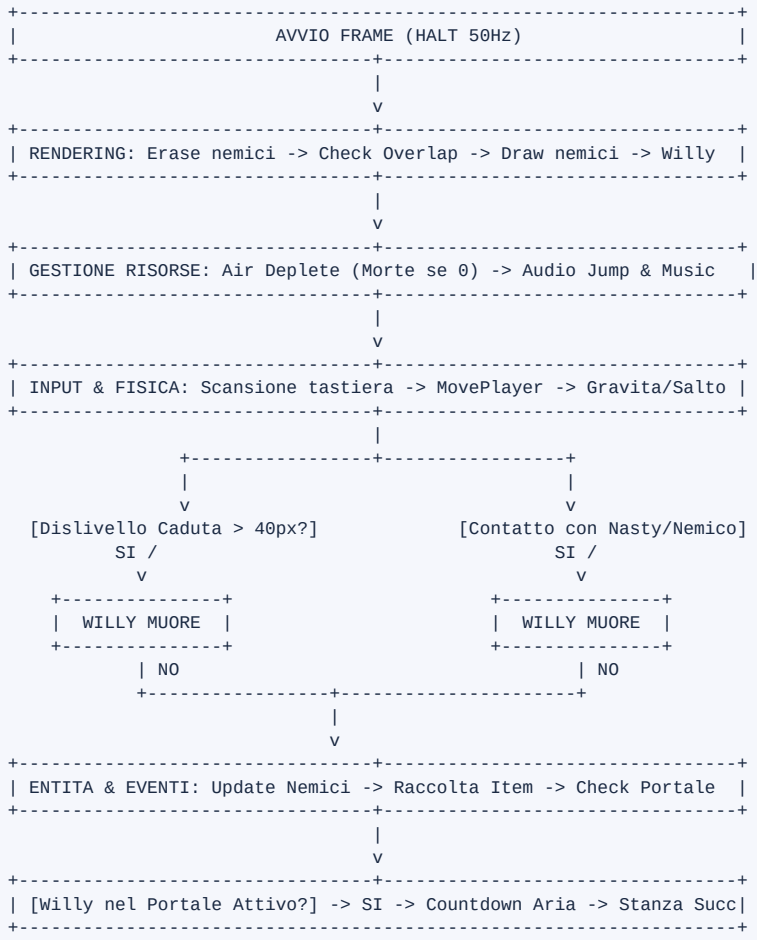
items.asm - Score_AddItemPoints

```
Score_AddItemPoints:
    ; Incrementa di 100 punti (Byte 1 = centinaia/migliaia)
    ld   a, (player_score + 1)
    add  a, $01
    daa                  ; Decimal Adjust Accumulator hardware!
    ld   (player_score + 1), a
    ; Propagazione eventuale riporto al Byte 2 (decine/centinaia di migliaia)
    ld   a, (player_score + 2)
    adc  a, 0
    daa                  ; Decimal Adjust per il riporto!
    ld   (player_score + 2), a
    call Score_CheckAndUpdateHighScore
    call Gfx_UpdateScoreDisplay
    ret
```

12 DIAGRAMMI DI FLUSSO E SCHEMI ARCHITETTURALI

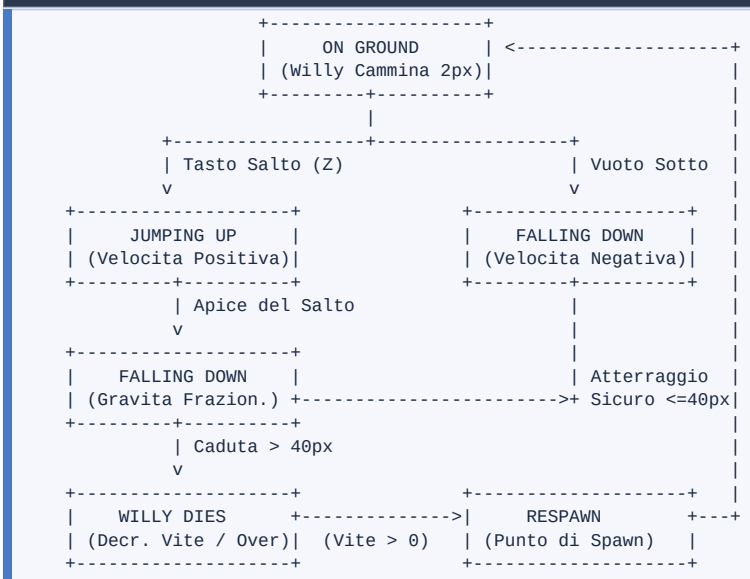
12.1 Flusso del Game Loop Principale

FLOWCHART COMPLETO DEL GAME LOOP A 25 FPS



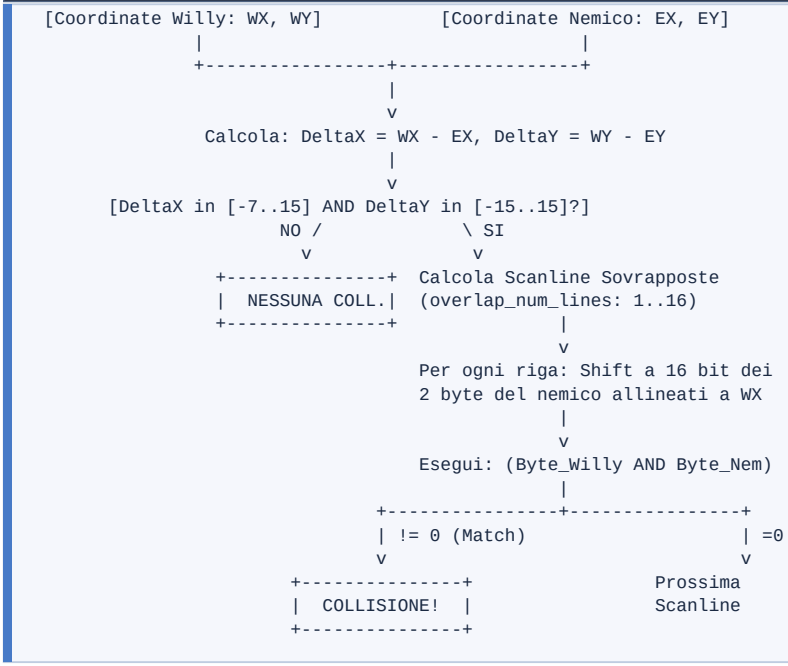
12.2 Macchina a Stati di Miner Willy

MACCHINA A STATI FINITI DI MINER WILLY



12.3 Pipeline di Collision Detection Pixel-Perfect

ALGORITMO BIT-A-BIT DI COLLISION DETECTION



13 ANALISI CRITICA: WORKAROUND, LIMITAZIONI E PUNTI DELICATI

13.1 Gestione dell'Attribute Clash dello ZX Spectrum

Il limite hardware piu celebre dello ZX Spectrum e' l'Attribute Clash (conflitto di colore): ogni cella di 8x8 pixel puo contenere un solo colore di inchiostro (INK) e un solo colore di sfondo (PAPER).

> **Soluzione Adottata:** Nel motore di Manic Miner, gli sprite dinamici (Willy e i guardiani) non alterano permanentemente la memoria attributi dello sfondo. Quando un guardiano si muove, sovrascrive temporaneamente l'attributo delle celle che attraversa; al momento della cancellazione (Gfx_RestoreEnemyColorAttributes), l'attributo originale della caverna viene ripristinato.

> **Willy in Primo Piano:** Per evitare che Willy assuma il colore del nemico quando si incrociano, Willy viene sempre ridisegnato DOPO i guardiani, imponendo il suo inchiostro (Cyan su sfondo trasparente) al di sopra dell'attributo del nemico.

13.2 Risoluzione del Glitch di Aggancio Rampe dal Basso (ramp_entry_guard)

Nelle prime versioni dei platform su Spectrum, l'equazione lineare di una rampa calcolata unicamente in base a X (Y = BaseY - X) provocava un grave glitch: se il giocatore camminava sul pavimento sottostante la rampa, il motore interpretava la sua coordinata X come appartenente alla rampa e lo 'teletrasportava' magneticamente in alto sulla superficie inclinata.

> **Implementazione della Guardia Orizzontale:** La variabile ramp_entry_guard impedisce l'aggancio quando Willy entra orizzontalmente in una stanza con una rampa finche non si allontana dalla zona di confine o non e' gia alla quota corretta.

> **Controllo di Provenienza Verticale in Salto:** Durante la salita in salto, la quota iniziale jump_start_y viene confrontata con la quota teorica della rampa ramp_Y: se Willy proveniva da sotto la rampa (jump_start_y > ramp_Y), l'atterraggio viene bloccato permettendo a Willy di passare oltre senza agganci impropri.

13.3 Preservazione dei Registri in Funzioni di Calcolo Fisico

Un punto delicatissimo emerso dall'analisi dei sorgenti riguarda la gestione dei registri CPU nelle routine matematiche:

> **Il Caso GetRampY e Registro D:** GetRampY calcola la quota della rampa in A e clobbera il registro D durante le sottrazioni. La routine chiamante falling_checks utilizzava D per conservare old_Y: un confronto tra D e la quota rampa generava un bug per cui Willy atterrava sulla rampa anche provenendo da sotto. Il codice attuale risolve elegantemente rileggendo old_Y direttamente dalla memoria (ld a, (PlayerPos+1)).

13.4 Gestione dello Stack Pointer (ld sp, $fdf0) per Evitare Stack Drift

In un gioco complesso con salti diretti, transizioni tra livelli e respawn continui, un uso disinvolto di chiamate CALL senza corrispondenti RET puo provocare uno 'Stack Drift' (progressivo riempimento dello stack con conseguente crash per sovrascrittura delle variabili in RAM):

> **Workaround Hardware Rigoroso:** All'inizio di LoadRoomAndRestart, ShowTitleScreen, TriggerGameOver e dei salti di warp stanza, il codice esegue un reset esplicito e incondizionato dello Stack Pointer: ld sp, $fdf0. Questo garantisce che lo stack torni sempre alla sua posizione vergine in cima alla memoria, rendendo il programma immune a qualsiasi stack overflow.

13.5 Limiti di Scalabilita del Modello a 1024 Byte

Sebbene la struttura rigida a 1024 byte per stanza sia un capolavoro di economia per l'hardware del 1983, presenta limiti architetturali determinati dai vincoli dei buffer fissi:

> **Massimo 5 Oggetti per Caverna:** Lo spazio riservato a OFFSET_ITEMS (26 byte) consente un massimo di 5 record da 5 byte piu terminatore $FF.

> **Massimo 4 Guardiani Orizzontali:** Lo spazio a OFFSET_H_GUARDIANS (29 byte) ospita al massimo 4 guardiani da 7 byte piu terminatore $FF.

> **Risoluzione Fissa a Schermata Singola:** Non e' previsto scrolling multidirezionale: ogni caverna e' rigorosamente confinata nella matrice 16x32 blocchi.

CONCLUSIONE DELL'ANALISI TECNICA

L'architettura del sorgente di Manic Miner dimostra un'eccezionale maestria nell'ingegneria del software su Z80. I vincoli fisici estremi dello ZX Spectrum 48K sono stati superati non con compromessi qualitativi, ma attraverso algoritmi deterministici brillanti: decompressione al volo, collision detection bit-a-bit a 16 bit, aritmetica BCD hardware e gestione atomica della VRAM.